Computational Chronobiology and Combinatorial Rescheduling in High-Stakes Academic Time-Blocking Engines

Chronobiology and Cognitive Load Optimization

Empirical Neurobiological Frameworks and Task Mapping

Human cognitive throughput oscillates systematically across the 24-hour day under the coupled influence of two primary physiological mechanisms formalized in the Two-Process Model of Sleep Regulation: the homeostatic sleep drive (Process S) and the endogenous circadian pacemaker (Process C). Extended biomathematical models, most notably the Sleep, Activity, Fatigue, and Task Effectiveness (SAFTE) framework, demonstrate that neurobehavioral capacity is not static but fluctuates as a continuous function of prior wakefulness, circadian phase, and transient sleep inertia.

Process S represents the progressive accumulation of neurochemical somnogens, primarily extracellular adenosine in the basal forebrain and cerebral cortex, generated as a metabolic byproduct of continuous neuronal activity and astrocytic glycogen consumption. During wakefulness, this homeostatic pressure rises asymptotically toward an upper saturation threshold according to the first-order exponential equation:

$$S(t) = \mu + (S_0 - \mu) e^{-\frac{t - t_0}{\chi_w}}$$

where μ represents the upper asymptote of homeostatic sleep debt, S_0 is the baseline homeostatic state at wake onset t_0, and $\chi_w \approx 18.2\text{ hours}$ is the characteristic waking accumulation time constant. During restorative slow-wave sleep (NREM Stage N3), adenosine is systematically cleared through astrocytic uptake and glymphatic exchange, causing Process S to decay exponentially toward zero:

$$S(t) = S_0 e^{-\frac{t - t_0}{\chi_s}}$$

with an operational clearance time constant $\chi_s \approx 4.2\text{ hours}$.

Process C represents an exogenous-independent oscillatory drive generated by the master circadian pacemaker within the suprachiasmatic nucleus (SCN) of the anterior hypothalamus. The SCN coordinates systemic circadian rhythms via downstream autonomic pathways and endocrine cascades, tracking closely with core body temperature (CBT) and the hypothalamic-pituitary-adrenal (HPA) axis. In adolescent and young adult populations, an ontological biological phase delay occurs, shifting the circadian phase by 1.5 to 3 hours relative to mature adults. This delay is mediated by both a lengthened intrinsic circadian period and altered retinal sensitivity to evening blue light. Consequently, the adolescent core body temperature minimum ($T_{\min}$) typically occurs between 05:00 and 07:00, in contrast to 03:30 to 05:00 in mature cohorts.

The circadian alerting signal can be modeled computationally using the dual-harmonic trigonometric function employed in the SAFTE framework:

$$C(t) = \cos\left(\frac{2\pi(t - p)}{24}\right) + \beta \cos\left(\frac{4\pi(t - p - p')}{24}\right)$$

where p defines the fundamental 24-hour circadian phase peak in hours, p' is the relative phase shift of the secondary harmonic, and \beta \approx 0.5 designates the relative amplitude of the 12-hour circasemidian wave.

In addition to homeostatic and circadian drives, the nervous system experiences sleep inertia (Process W), a temporary period of hypo-arousal and hypofrontality upon awakening caused by delayed microvascular perfusion and the gradual reactivation of the prefrontal-thalamocortical loops:

$$W(t) = W_0 e^{-\frac{t - t_{\text{wake}}}{\tau_{\text{inertia}}}}$$

where $W_0 < 0$ is the maximal initial negative deficit and $\tau_{\text{inertia}} \approx 0.5\text{ hours}$ (30 minutes) represents the recovery time constant.

Integrating these physiological variables yields the net neurocognitive effectiveness function:

$$A(t) = w_c C(t) - w_s S(t) + W(t)$$

where w_c and w_s are empirically derived psychomotor vigilance scaling coefficients.

Superimposed upon this macro-circadian baseline is Kleitman's Basic Rest-Activity Cycle (BRAC), an endogenous ultradian oscillation ranging between 90 and 120 minutes. Governed by alternating autonomic sympathetic and parasympathetic dominance, the BRAC drives cyclic shifts in cortical activation. During the peak of an ultradian cycle, high-frequency EEG activity (beta and gamma bands) predominates, optimizing prefrontal signal-to-noise ratios, focused attention, and working memory performance. As the cycle moves into its trough, slower electrophysiological rhythms (theta-band activity) emerge, corresponding to reduced attentional focus, lapses in working memory, and temporary cognitive fatigue.

Metric / Parameter	Value / Equation	Physiological Mechanism	Operational Scheduling Meaning
Process S Build-up	$\chi_w \approx 18.2\text{ h}$	Extracellular adenosine accumulation in the basal forebrain	Sets an upper limit on continuous daily wakefulness before severe cognitive deficits emerge
Process S Clearance	$\chi_s \approx 4.2$[span_18](start_span)[span_18](end_span) h	Glymphatic clearance and SWA during NREM stage N3	Governs minimum sleep duration needed to reset baseline cognitive efficiency
Adolescent $T_{\min}$ Phase	05:00 – 07:00	Pubertal delay in SCN firing and melatonin release profile	Shifts peak analytical focus windows 2–3 hours later than adult baselines
Circasemidian Harmonic (β)	$\beta \approx 0.5$, period = 12 h[span_25](start_span)[span_25](end_span)[span_30](start_span)[span_30](end_span)	Endogenous secondary SCN rhythm coupled with CBT inflection	Generates the universal mid-afternoon drop in cognitive throughput
BRAC Periodicity	$90 - 120\text{ minutes}$	Ultradian brainstem reticular-thalamocortical oscillatory pacing	Defines natural boundaries for sustained, uninterrupted study blocks

Metric / Parameter	Value / Equation	Physiological Mechanism	Operational Scheduling Meaning
Sleep Inertia (τ_{inertia})	$\tau \approx 30\text{ minutes}$	Prefrontal hypoperfusion and slow clearance of adenosine upon waking	Prevents high-load analytical task execution immediately following wake-up

Because different academic tasks place distinct demands on neural substrates, scheduling algorithms must align specific task categories with corresponding neurobiological windows:
Analytical tasks—such as mathematical proof construction, physics derivation, formal code synthesis, and structured algorithmic reasoning—rely heavily on central executive networks, the dorsolateral prefrontal cortex (dlPFC), and robust working memory buffers. These tasks require low psychomotor reaction-time variability, optimal catecholaminergic tone (dopamine and norepinephrine), and minimal adenosine interference. Consequently, analytical work is best scheduled during the morning circadian ascent ($T_{\min} + 3.5\text{h}$ to $T_{\min}$ +[span_19](start_span)[span_19](end_span) 6.5h, or 09:00 to 12:30 for adolescents), where rising core body temperature coincides with the clearance of the cortisol awakening response and low levels of Process S.
Synthesizing tasks—such as comparative literary analysis, historical historiography, argumentative essay composition, and conceptual hypothesis formulation—draw upon broader semantic networks, bilateral fronto-temporal connections, and flexible cognitive associations. These tasks benefit from divergent thinking, where slight cognitive disinhibition allows for novel semantic connections without losing the executive control needed for structural coherence. This cognitive state corresponds to the late-afternoon circadian alertness plateau ($T_{\min} + 10.5\text{h}$ to $T_{\min} + 13.5\text{h}$, or 16:30 to 19:30 for adolescents), where peak core body temperature supports rapid information processing while moderate adenosine levels reduce overly rigid top-down inhibition.
Administrative tasks—such as filing course documents, reading procedural syllabi, basic flashcard tagging, and sorting academic reference metadata—involve automated motor-perceptual pathways and place minimal demands on the central executive network. Because these activities remain robust even during degraded cognitive states, they can be scheduled during periods of elevated sleep pressure or circadian lows without impacting work quality.

Mathematical Penalization of the Post-Prandial Circadian Dip

Between 13:30 and 15:30 ($T_{\min} + 7.5\text{h}$ to $T_{\min} + 9.5\text{h}$ in the adolescent phase), human cognitive architecture encounters the post-prandial circadian dip. While often assumed to be caused entirely by digestion, forced desynchrony and constant routine protocols demonstrate that this drop occurs independently of nutrient ingestion, driven directly by the circasemidian secondary harmonic of the central circadian pacemaker.
Digestive processes do, however, compound this drop: carbohydrate-heavy meals prompt rapid shifts in blood glucose, transient parasympathetic dominance, and the release of gastrointestinal peptide hormones that cross the blood-brain barrier to promote somnolence. This combination produces decreased psychomotor vigilance, increased reaction-time variance, and lapses in sustained prefrontal attention.
To prevent high-load tasks from being placed during this window, the scheduling engine applies an objective penalty function, $P_{\text{circ}}(i)$, to any task i allocated within the time domain

$[t_{\text{start}}, t_{\text{end}}]$:
$$P_{\text{circ}}(i) = \int_{t_{\text{start}}}^{t_{\text{end}}} \omega_{\text{task}} \cdot \kappa_i \cdot \Psi_{\text{dip}}(t) \, dt$$
The cognitive weight $\omega_{\text{task}} \in [0.0, 1.0]$ scales with the mental load of the task category: $\omega_{\text{analytical}} = 1.0$, $\omega_{\text{synthesizing}} = 0.60$, and $\omega_{\text{administrative}} = 0.05$. The parameter κ_i acts as a priority multiplier for the individual task.
The circadian dip penalty kernel, $\Psi_{\text{dip}}(t)$, is defined continuously using a modified Gaussian distribution centered over the physiological nadir:
$$\Psi_{\text{dip}}(t) = \exp\left(-\frac{(t - t_{\text{nadir}})^2}{2\sigma_{\text{dip}}^2} \right)$$
where t_{nadir} represents the user's empirical dip center (defaulting to 14:15, or $T_{\min} + 8.25\text{h}$) and $\sigma_{\text{dip}} \approx 0.75\text{ hours}$ (45 minutes) defines the distribution's spread, concentrating the penalty across the 13:30 to 15:30 window.
When evaluated within discrete time-blocks of duration $\Delta t = t_{\text{end}} - t_{\text{start}}$, the system applies the numerical midpoint approximation:
$$P_{\text{circ}}(i) \approx \omega_{\text{task}} \cdot \kappa_i \cdot \exp\left(-\frac{(\bar{t}_i - t_{\text{nadir}})^2}{2\sigma_{\text{dip}}^2} \right) \cdot \Delta t$$
where $\bar{t}_i = \frac{t_{\text{start}} + t_{\text{end}}}{2}$.
This penalty functions as a cost factor added to the combinatorial objective function. As a result, placing a 90-minute analytical STEM problem set within this dip incurs an exponential penalty, steering the optimization engine to reserve this period for deliberate physical rest, mild aerobic activity, or low-intensity administrative tasks.

Quantitative Physiological and Cognitive Thresholds

Designing effective schedules for high-intensity pre-university and International Baccalaureate students requires defining clear numerical limits for cognitive effort and recovery:

Maximum Sustained Deep Work Block (75 to 90 Minutes)

Sustained directed attention consumes intracellular glycogen reserves and increases the concentration of extracellular glutamate within prefrontal cortical synapses. Concurrently, Kleitman's BRAC introduces an ultradian fatigue boundary at roughly 90 minutes. Psychomotor vigilance data demonstrate that after 75 minutes of continuous high-load computational effort, cognitive efficiency drops significantly. Without a break, lapse rates (reaction times exceeding 500 milliseconds) increase rapidly, and working memory retention declines.
This drop in cognitive efficiency, $\eta_{\text{eff}}(t)$, can be modeled as a logistic decay function:
$$\eta_{\text{eff}}(t) = \frac{1}{1 + \exp\left(k_{\text{decay}} \cdot (t - t_{\text{crit}})\right)}$$
where $t_{\text{crit}} = 80\text{ minutes}$ and $k_{\text{decay}} = 0.08\text{ min}^{-1}$.
At $t = 90\text{ minutes}$, cognitive efficiency drops to approximately 31\%, falling well below the acceptable 50\% operational threshold ($\eta_{\text{eff}} < 0.50$). Consequently, individual deep work blocks must be capped at a maximum of 90 minutes, with an optimal target duration of 75 minutes.

Maximum Total Daily Deep Work Volume (4.0 to 4.5 Hours)

The deliberate practice research pioneered by Ericsson and corroborated by neuroimaging

studies of allostatic load shows that exceptional performers across complex cognitive domains rarely sustain more than 4 to 4.5 hours of high-concentration deliberate practice per day.
In developing adolescents, the prefrontal cortex is actively maturing through extensive synaptic pruning and myelination, making it vulnerable to chronic stress and neuroendocrine strain.
Beyond 4.5 hours of intense, focused cognitive work, marginal learning returns drop toward zero, and the system begins accumulating severe allostatic debt.
Sustained work beyond this ceiling depletes homeostatic reserves without yielding meaningful memory consolidation, elevates evening cortisol levels, and disrupts subsequent slow-wave sleep cycles, degrading performance on following days.

Minimum Physical/Recovery Buffer Post-School (45 to 60 Minutes)

Secondary school timetables require sustained, externally driven attention across multiple subjects, producing Directed Attention Fatigue (DAF). According to Kaplan's Attention Restoration Theory (ART), restoring directed attention mechanisms requires disengaging the central executive network and activating the default mode network (DMN) alongside involuntary attention mechanisms ("soft fascination").
At the cellular level, this rest period clears accumulated metabolic waste and restores baseline neurotransmitter balances in prefrontal networks.
A recovery window of 45 to 60 minutes must therefore separate the end of the school day from the start of evening independent study. Shortening this buffer to under 30 minutes leads to premature cognitive exhaustion and poor focus during evening study sessions.

Combinatorial Optimization and Algorithmic Handling of Schedule Drift

Algorithmic Foundations in Real-Time Operations Research

Managing real-time schedule drift can be modeled as a single-machine preemptive scheduling problem with dynamic release dates, sequence-dependent transition costs, and hard deadline constraints.

Earliest Deadline First with Virtual Deadlines (EDF-VD)

Earliest Deadline First (EDF) is an optimal dynamic scheduling algorithm on a single processor, provided total workload utilization does not exceed capacity ($U \le 1.0$).
However, human academic schedules experience dynamic over-utilization when activities take longer than planned.
To prevent sudden cascade failures across all deadlines, Mixed-Criticality systems use Virtual Deadlines (EDF-VD). High-priority academic obligations (such as an immutable IB internal assessment deadline) receive an artificial, earlier virtual deadline:
$$\hat{D}_i = r_i + \lambda_{\text{crit}} (D_i - r_i)$$
where r_i is the task release time, D_i is the absolute external deadline, and $\lambda_{\text{crit}} \in (0, 1)$ is a tuning parameter based on the relative utilization of high- versus low-criticality tasks.
When unexpected delays occur, the engine uses these virtual deadlines to prioritize critical commitments, ensuring that low-priority tasks yield well before high-priority deadlines are

compromised.

Elastic Task Model (Buttazzo et al.)

The Elastic Task Model treats tasks like mechanical springs that compress when system load increases. Each task τ_i is defined by a nominal execution duration d_i^0, a minimum allowable duration $d_i^{\min}$, and an elasticity coefficient $e_i \ge 0$.
The parameter e_i reflects the task's adaptability: tasks with $e_i = 0$ are completely rigid and cannot be shortened, whereas tasks with high e_i values absorb reductions readily.
When the total requested duration exceeds the available time budget C_{avail} by an excess deficit $\Delta_{\text{excess}} = \sum d_i^0 - C_{\text{avail}}$, the engine calculates compressed task durations:
$$d_i = d_i^0 - \left(\frac{e_i}{\sum_{j \in V} e_j}\right) \Delta_{\text{excess}}$$
Chantem et al. showed that this formulation corresponds to solving a constrained Quadratic Programming problem that minimizes the sum of squared deviations from each task's nominal duration:
$$\min \sum_{i=1}^n \frac{(d_i^0 - d_i)^2}{e_i} \quad \text{subject to} \quad \sum_{i=1}^n d_i \le C_{\text{avail}}, \quad d_i^{\min} \le d_i \le d_i^0$$
This quadratic formulation distributes compression across all elastic tasks, preventing single tasks from bearing the entire burden of a schedule delay.

Critical Chain Buffer Management (CCPM) and RSEM Sizing

Traditional planning methods often fail when individuals fall prey to Parkinson's Law ("work expands to fill the time available") or Student Syndrome ("procrastinating until the deadline looms").
Critical Chain Project Management (CCPM) addresses these behavioral tendencies by removing individual safety margins from single tasks, setting aggressive baseline estimates (t_{50}, representing a 50% probability of on-time completion), and aggregating protection into pooled time buffers.
The Root Square Error Method (RSEM) calculates the size of these protective buffers using the Central Limit Theorem:
$$\text{Buffer}_{\text{RSEM}} = \sqrt{\sum_{i=1}^n (t_{90, i} - t_{50, i})^2}$$
where $t_{90, i}$ is the conservative duration estimate (90% completion certainty) and $t_{50, i}$ is the aggressive median estimate.
By pooling uncertainty into dedicated feeding buffers, the system absorbs task drift without constantly disrupting downstream commitments.

Deterministic Rescheduling Heuristic: The 45-Minute Mid-Day Slippage

Consider a real-world scenario where a student falls behind schedule by $\Delta_{\text{slip}} = 45\text{ minutes}$ at 14:00. The remainder of the daily schedule contains three distinct categories of time-blocks:

- Fixed commitments (rigid constraints, $e_i = 0$, such as an in-person chemistry lab at 16:00).
- Soft milestone tasks (elastic tasks, $e_i > 0$, such as problem sets, essay reading, or topic

review).
- Centralized buffers (explicit unallocated slack and feeding buffers).

The deterministic repair algorithm resolves this delay through five structured phases:

1. **Deficit Evaluation:** The engine determines the net schedule deficit \Delta_{\text{slip}} = t_{\text{actual}} - t_{\text{expected}} = 45\text{ minutes} and scans the timeline up to the next fixed boundary (T_{\text{fixed}} = 16:00). It measures total unallocated slack S_{\text{free}} within this window. If S_{\text{free}} \ge \Delta_{\text{slip}}, the engine shifts intermediate soft tasks into the slack space, resolving the delay without altering task durations.
2. **Buffer Consumption (CCPM Phase):** If S_{\text{free}} < \Delta_{\text{slip}}, the engine draws from scheduled feeding buffers within the current operational window, reducing the active deficit to:\Delta_{\text{deficit}} = \Delta_{\text{slip}} - S_{\text{free}} - \Delta_{\text{buffer_consumed}}
3. **Elastic Compression (Buttazzo Mechanics):** If a deficit remains (\Delta_{\text{deficit}} > 0), the engine identifies all compressible soft tasks within the active window, V_{\text{soft}}. It calculates their combined compression capacity:M_{\text{comp}} = \sum_{j \in V_{\text{soft}}} (d_j^0 - d_j^{\min}) If M_{\text{comp}} \ge \Delta_{\text{deficit}}, the remaining deficit is absorbed by compressing tasks according to their elasticity coefficients:d_j^{\text{new}} = d_j^0 - \Delta_{\text{deficit}} \cdot \left(\frac{e_j}{\sum_{k \in V_{\text{soft}}} e_k} \right) If any task reaches its lower threshold (d_j^{\text{new}} < d_j^{\min}), the engine locks that task to its minimum duration d_j^{\min}, removes it from V_{\text{soft}}, and recalculates the remaining compression across the remaining tasks in V_{\text{soft}}.
4. **Selective Deferral / Dropping (Knapsack Elimination):** If M_{\text{comp}} < \Delta_{\text{deficit}}, the deficit exceeds available compression limits. The engine locks all elastic tasks to their minimum values (d_j = d_j^{\min}) and computes the remaining deficit:\Delta_{\text{remain}} = \Delta_{\text{deficit}} - M_{\text{comp}} The engine then removes low-priority tasks to clear the remaining deficit, solving a 0/1 Knapsack minimization problem over the candidate set:\min \sum_{k \in V_{\text{drop}}} w_k \quad \text{subject to} \quad \sum_{k \in V_{\text{drop}}} d_k^{\min} \ge \Delta_{\text{remain}} where w_k represents the task's subjective priority weight. Dropped tasks are automatically moved to the unscheduled backlog for future planning.
5. **Timeline Regeneration:** The engine iterates through the adjusted task list, setting each start time to:t_{\text{start}, i} = \max(t_{\text{current}}, t_{\text{end}, i-1}) with end times t_{\text{end}, i} = t_{\text{start}, i} + d_i. The immovable fixed boundary at 16:00 remains untouched, and the updated schedule is restored to a feasible state.

Deterministic Rescheduling Engine Implementation

The following Kotlin implementation provides a deterministic, solver-free rescheduling algorithm designed to execute locally on low-power mobile devices in O(N^2) time:

```
data class TimeBlock(
    val id: String,
    val name: String,
    var startMinutes: Int,
    var durationMinutes: Int,
    val minDurationMinutes: Int,
    val isFixedCommitment: Boolean,
```

```
    val elasticity: Double,       // e_i: Relative elasticity
coefficient
    val priorityWeight: Double     // w_i: Relative utility value
)

class DeterministicReschedulingEngine {

    fun handleScheduleSlippage(
        timeline: MutableList<TimeBlock>,
        currentTimeMinutes: Int,
        slippageMinutes: Int
    ): List<TimeBlock> {
        var deficit = slippageMinutes

        // 1. Separate historical blocks from downstream candidates
        val pastBlocks = timeline.filter {
            (it.startMinutes + it.durationMinutes) <=
currentTimeMinutes
        }
        val futureBlocks = timeline.filter {
            (it.startMinutes + it.durationMinutes) >
currentTimeMinutes
        }.sortedBy { it.startMinutes }.toMutableList()

        if (futureBlocks.isEmpty()) return pastBlocks

        // 2. Identify the first immovable temporal constraint (Fixed
Commitment)
        val fixedIndex = futureBlocks.indexOfFirst {
it.isFixedCommitment }
        val activeWindow = if (fixedIndex == -1) {
            futureBlocks
        } else {
            futureBlocks.subList(0, fixedIndex)
        }

        // 3. Absorb drift using available free slack in the active
window
        for (i in 0 until activeWindow.size - 1) {
            val currentBlockEnd = activeWindow[i].startMinutes +
activeWindow[i].durationMinutes
            val nextBlockStart = activeWindow[i + 1].startMinutes
            val slack = nextBlockStart - currentBlockEnd

            if (slack > 0) {
                val absorbed = kotlin.math.min(slack, deficit)
                deficit -= absorbed
                if (deficit == 0) break
```

```
            }
        }

        // 4. Compress elastic tasks proportionally using Buttazzo
mechanics
        if (deficit > 0) {
            val compressibleTasks = activeWindow.filter {
                !it.isFixedCommitment && it.durationMinutes >
it.minDurationMinutes
            }.toMutableList()

            while (deficit > 0 && compressibleTasks.isNotEmpty()) {
                val totalElasticity = compressibleTasks.sumOf {
it.elasticity }
                var clampedAny = false

                for (task in compressibleTasks.toList()) {
                    val proportionalCut = kotlin.math.round(
                        deficit * (task.elasticity / totalElasticity)
                    ).toInt()

                    val maxAllowableReduction = task.durationMinutes -
task.minDurationMinutes
                    val actualReduction =
kotlin.math.min(proportionalCut, maxAllowableReduction)

                    if (actualReduction > 0) {
                        task.durationMinutes -= actualReduction
                        deficit -= actualReduction
                    }

                    if (task.durationMinutes ==
task.minDurationMinutes) {
                        compressibleTasks.remove(task)
                        clampedAny = true
                        break
                    }
                    if (deficit <= 0) break
                }

                // Fallback: incrementally compress the remaining
tasks if rounding leaves a deficit
                if (!clampedAny && deficit > 0 &&
compressibleTasks.isNotEmpty()) {
                    val fallbackTask = compressibleTasks.first()
                    val reduction = kotlin.math.min(
                        fallbackTask.durationMinutes -
fallbackTask.minDurationMinutes,
```

```
                        deficit
                    )
                    fallbackTask.durationMinutes -= reduction
                    deficit -= reduction
                    if (fallbackTask.durationMinutes ==
fallbackTask.minDurationMinutes) {
                        compressibleTasks.remove(fallbackTask)
                    }
                }
            }
        }

        // 5. Defer lowest-priority tasks if a deficit remains
        if (deficit > 0) {
            val candidatesForDrop = activeWindow.filter {
!it.isFixedCommitment }
                .sortedBy { it.priorityWeight /
it.durationMinutes.toDouble() }

            for (task in candidatesForDrop) {
                val recoveredTime = task.durationMinutes
                futureBlocks.remove(task) // Defer task back to
backlog
                deficit -= recoveredTime
                if (deficit <= 0) break
            }
        }

        // 6. Reconstruct the execution timeline without overlaps
        var cursor = currentTimeMinutes
        for (block in futureBlocks) {
            if (block.isFixedCommitment) {
                if (cursor > block.startMinutes) {
                    // Critical overrun: push the hard boundary
forward
                    block.startMinutes = cursor
                }
                cursor = block.startMinutes + block.durationMinutes
            } else {
                block.startMinutes = cursor
                cursor += block.durationMinutes
            }
        }

        return pastBlocks + futureBlocks
    }
}
```

Spaced-Repetition and Reverse Milestone Back-Planning

Empirical Retention Dynamics and Spacing Ratios

Long-range academic deliverables—such as multi-month research papers or cumulative final examinations—require distributing preparation across structured intervals to promote lasting retention.

Cepeda et al. (2008) mapped these memory dynamics in a large-scale study of over 1,350 participants across 26 experimental conditions, analyzing the relationship between Inter-Study Intervals (ISI), Retention Intervals (RI), and recall accuracy.

Their findings demonstrate an asymmetric, inverted U-shaped relationship between the ratio $\phi = \frac{ISI}{RI}$ and final test retention:

Target Retention Interval (RI)	Empirically Optimal Spacing Ratio ($\phi = \frac{ISI}{RI}$)	Optimal Inter-Study Interval (ISI)	Effect of Under-Spacing ($ISI < ISI_{\text{opt[span_94](start_span)[span_94](end_span)[span_103](start_span)[span_103](end_span)}}$)	Effect of Over-Spacing ($ISI > ISI_{\text{opt}}$)
7 Days (1 Week)	20\% - 40\%	1.4 - 2.8\text{ days}	Rapid decay; low desirable difficulty	Minor decline; near-baseline retention
30 Days (1 Month)	10\% - 20\%	3.0 - 6.0\text{ days}	Moderate retention loss; illusory fluency	Gradual forgetting; increased retrieval effort
84 Days (3 Months)	8\% - 15\%	6.7 - 12.6\text{ d[span_120](start_span)[span_120](end_span)ays}	High decay; resembles massed cramming	Memory trace drops below retrieval threshold
350 Days (1 Year)	5\% - 10\%	17.5 - 35.0\text{ days}	Severe long-term attrition; weak stability	Total trace decay; requires relearning

These empirical data reveal that the optimal spacing ratio ϕ decreases systematically as the target retention interval expands. When planning for an exam occurring one year later, review sessions should not be scheduled at the 20% to 40% ratios optimal for weekly retention, as this leads to inefficiently frequent early sessions and unmanaged decay later in the year.

To model retrievability over time, modern spaced learning frameworks use an exponential retention function:

$$R(t) = \exp\left(-\frac{t}{S} \right)$$

where S represents memory stability (the time in days for recall probability to decline to $e^{-1} \approx 36.8\%$) and t is the time elapsed since the last retrieval event.

When a retrieval session occurs at a point of desirable difficulty (typically when R declines to between 0.85 and 0.90), memory stability expands according to a non-linear multiplier:
$$S_{k+1} = S_k \cdot \left(1 + \alpha \cdot D^{-\beta} \cdot \exp(1 - R)\right)$$
where D denotes content difficulty, while α and β are subject-specific learning constants.
This non-linear scaling provides the mathematical basis for expanding review intervals, widening each successive interval as memory stability increases.

Reverse Milestone Back-Planning Logic

For major academic projects, the engine applies reverse milestone back-planning across two coordinated planning tracks: concrete deliverable milestones (such as writing sections or experimental procedures) and spaced retrieval revision nodes. The planning workflow runs backward from the terminal deadline:
The process begins by anchoring the terminal completion date (T_{final}) as an immutable boundary. The engine then works backward to schedule preceding deliverable stages (M_k). Each stage is assigned a required working duration based on estimated workload and bounded by an RSEM-calculated contingency buffer:
$$t_{M_{k-1}} = t_{M_k} - \text{Duration}(M_k) - \text{Buffer}_{\text{RSEM}}(M_k)$$
This reverse pass ensures that all major phases—such as completing a primary literature analysis, generating experimental drafts, or producing final proofs—are scheduled with appropriate safety margins ahead of the final delivery date.
In parallel, the engine schedules spaced revision sessions across the timeline. For a course syllabus introduced on day t_{initial} and tested on day T_{final}, the system schedules N distinct review sessions. Rather than using fixed linear intervals, review dates are calculated using an expanding geometric progression:
$$t_k = t_{\text{initial}} + (T_{\text{final}} - t_{\text{initial}}) \cdot \left(\frac{k}{N}\right)^\gamma$$
where $\gamma \approx 1.8$ introduces a geometric expansion that steadily widens the intervals between successive sessions, matching the lengthening stability intervals of long-term memory.

Preventing Timetable Congestion via Capacity Bounding

A common problem in spaced repetition engines is the "review avalanche," where accumulated daily review tasks overwhelm the student's available study time. To prevent this, the scheduling algorithm applies a strict daily capacity cap:
$$\sum_{j \in R_{\text{day}}} \text{Duration}(j) \le \Gamma_{\text{cap}} \cdot C_{\text{avail}}(t)$$
The maximum capacity ratio Γ_{cap} is capped at 0.20, ensuring that spaced retrieval tasks never consume more than 20% of the student's daily independent study capacity $C_{\text{avail}}(t)$. The remaining 80% is preserved for core coursework, ongoing classroom assignments, and deliberate recovery.
When a newly generated review session conflicts with this capacity constraint, the engine resolves the bottleneck without omitting the review:

1. **Jitter Window Search:** The system defines a flexible scheduling window around the ideal review date:$[t_k - \delta_{\text{jitter}}, \, t_k + \delta_{\text{jitter}}]$ where $\delta_{\text{jitter}} = \lfloor 0.15 \cdot ISI_k \rfloor$. Because memory consolidation remains stable across minor timing shifts within this 15% window, moving the session does not impair long-term retention.
2. **Congestion Balancing:** Within this window, the engine places the review session on the

day with the lowest existing cognitive load:$t_{\text{assigned}} = \arg\min_{t \in [t_k \pm \delta_{\text{jitter}}]} \left(\frac{\text{ScheduledLoad}(t)}{C_{\text{avail}}(t)} \right)$

3. **Elastic Format Compression:** If every day within the jitter window is fully booked, the engine compresses the review task's duration using the Buttazzo spring model. The task transitions from an extended, open-ended problem session into a concise, high-yield retrieval format (e.g., shifting from broad synthesis problems to targeted active recall flashcards), preserving the memory retrieval event without exceeding daily capacity limits.

```
data class AcademicMilestone(
    val id: String,
    val title: String,
    val targetDeadlineEpochDay: Long,
    val estimatedEffortHours: Double
)

data class SpacedReviewBlock(
    val topicId: String,
    var scheduledEpochDay: Long,
    var durationMinutes: Int,
    val priorityWeight: Double
)

class SpacedRepetitionPlanner {

    fun generateExpandingReviewSchedule(
        topicId: String,
        startEpochDay: Long,
        examEpochDay: Long,
        dailyCapacityMinutes: Map<Long, Int>,
        maxReviewCapacityRatio: Double = 0.20
    ): List<SpacedReviewBlock> {
        val plannedReviews = mutableListOf<SpacedReviewBlock>()
        val totalHorizonDays = examEpochDay - startEpochDay
        if (totalHorizonDays <= 7) return plannedReviews

        // Determine session count based on total preparation horizon
        val sessionCount = when {
            totalHorizonDays > 180 -> 5
            totalHorizonDays > 60  -> 4
            totalHorizonDays > 21  -> 3
            else                   -> 2
        }

        val gamma = 1.8 // Expansion coefficient
        val nominalDurationMinutes = 45

        for (k in 1..sessionCount) {
            val progressRatio = k.toDouble() / sessionCount.toDouble()
            val dayOffset = kotlin.math.round(
```

```
                totalHorizonDays * Math.pow(progressRatio, gamma)
            ).toLong()
            val idealDay = startEpochDay + dayOffset

            val prevDay = if (k == 1) startEpochDay else
plannedReviews[k - 2].scheduledEpochDay
            val currentISI = idealDay - prevDay
            val allowableJitter = kotlin.math.max(1L, (currentISI *
0.15).toLong())

            val optimalAssignedDay = findLeastCongestedDay(
                targetDay = idealDay,
                jitter = allowableJitter,
                blockDuration = nominalDurationMinutes,
                capacityMap = dailyCapacityMinutes,
                existingReviews = plannedReviews,
                capacityRatioLimit = maxReviewCapacityRatio,
                lowerBoundDay = prevDay + 1,
                upperBoundDay = examEpochDay - 1
            )

            plannedReviews.add(
                SpacedReviewBlock(
                    topicId = topicId,
                    scheduledEpochDay = optimalAssignedDay,
                    durationMinutes = nominalDurationMinutes,
                    priorityWeight = 1.0 - (k * 0.12)
                )
            )
        }

        return plannedReviews
    }

    private fun findLeastCongestedDay(
        targetDay: Long,
        jitter: Long,
        blockDuration: Int,
        capacityMap: Map<Long, Int>,
        existingReviews: List<SpacedReviewBlock>,
        capacityRatioLimit: Double,
        lowerBoundDay: Long,
        upperBoundDay: Long
    ): Long {
        var selectedDay = targetDay
        var lowestLoadRatio = Double.MAX_VALUE

        val startScan = kotlin.math.max(lowerBoundDay, targetDay -
```

```
jitter)
        val endScan = kotlin.math.min(upperBoundDay, targetDay +
jitter)

        for (candidateDay in startScan..endScan) {
            val totalDailyCapacity = capacityMap[candidateDay] ?: 180
// Default 3 hours
            val ceilingMinutes = totalDailyCapacity *
capacityRatioLimit
            val currentScheduledMinutes = existingReviews
                .filter { it.scheduledEpochDay == candidateDay }
                .sumOf { it.durationMinutes }

            if (currentScheduledMinutes + blockDuration <=
ceilingMinutes) {
                return candidateDay // Direct placement available
            }

            val relativeLoad = currentScheduledMinutes.toDouble() /
ceilingMinutes
            if (relativeLoad < lowestLoadRatio) {
                lowestLoadRatio = relativeLoad
                selectedDay = candidateDay
            }
        }

        return selectedDay
    }
}
```

Psychological Friction, Behavioral Economics, and Abandonment Prevention

Cognitive Drivers of Calendar Abandonment

A primary reason students abandon deterministic time-blocking software is the "guilt loop," a psychological cascade triggered by rigid planning interfaces. This breakdown is driven by three well-documented behavioral and cognitive mechanisms:
The "What-the-Hell Effect" (counter-regulatory goal disinhibition), originally identified in self-regulation and dietary compliance research, occurs when an individual views a minor plan disruption as a complete failure of the entire system. When an unforeseen 30-minute delay disrupts a tightly scheduled calendar, the schedule transitions from an ordered state to a "broken" state. The perceived value of maintaining the rest of the schedule drops sharply, leading students to abandon their remaining planned tasks for the rest of the day and turn to escapist activities.

Visual debt and cognitive dissonance amplify this disengagement. Standard calendar interfaces highlight schedule slippage with high-contrast visual cues: tasks that are past due flash in bright red, error badges accumulate, and overdue banners highlight missed deadlines. Cognitive dissonance theory demonstrates that confronting visual evidence of unmet goals produces psychological discomfort. To avoid this negative feedback, users simply stop opening the application. The system transforms from an executive functioning support tool into a source of guilt and stress, prompting user disengagement.
The Planning Fallacy and systematic estimation bias compound this vulnerability. As Kahneman and Tversky established, individuals routinely construct forecasts based on best-case scenarios rather than historical reality. Secondary and pre-university students frequently build idealized schedules with back-to-back deep work sessions, allocating zero time for transition buffers, physiological fatigue, or cognitive context switching. When normal operational friction inevitably disrupts these unrealistic plans, the rigid system breaks down, triggering the guilt loop.

UX Architectural Paradigms for Sustained Adherence

To prevent abandonment, the software interface must treat schedule slippage as normal operating variance rather than user failure:

Emergency Reserves and Goal-Framed Slack

Sharif and Milkman (2021) demonstrated that framing schedule flexibility as an explicit "Emergency Reserve" significantly improves persistence compared to plans that are either completely rigid or vaguely defined.
When daily calendars include pre-allocated contingency buffers (such as two 30-minute flexible blocks or "emergency reserves"), drawing upon them to absorb delays does not register as a personal failure.
The user uses an intentionally designed system reserve, which preserves their perceived progress and maintains goal commitment.

Non-Punitive Rescheduling via Silent Auto-Healing

Visual interfaces should avoid punitive feedback states. Overrun tasks should never be rendered in red or marked with overdue warning indicators.
Instead, the system employs silent automated timeline healing:

- As soon as an ongoing task extends past its scheduled boundary, downstream elastic tasks automatically compress, and unallocated buffers absorb the overflow in real time.
- Completed tasks smoothly adjust their displayed duration to match actual logged time, while subsequent blocks glide forward without generating error states.
- The interface preserves a clear, achievable forward path, eliminating the need for manual schedule reorganizing.

Dynamic Calibration via Empirical Velocity Multipliers

Rather than relying on students to overcome the planning fallacy through discipline alone, the engine tracks their personal estimation variance over time:
$$\nu_{\text{history}} = \frac{\sum_{i=1}^m \text{Duration}_{\text{actual}}(i)}{\sum_{i=1}^m \text{Duration}_{\text{planned}}(i)}$$

When planning upcoming blocks, the engine transparently multiplies raw user estimates by this empirical correction factor:
$d_{\text{applied}} = d_{\text{user}} \cdot \max(1.0, \, \nu_{\text{history}})$
If a student consistently underestimates chemistry problem sets by 35% ($\nu_{\text{history}} = 1.35$), a requested 60-minute allocation is automatically scaled to 81 minutes behind the scenes.
The system builds resilience by adapting to observed user behavior rather than demanding unrealistic improvements in estimation accuracy.

Dimension	Rigid Traditional Time-Blocking	Adaptive Cognitive Scheduling Framework
Response to Delays	Leaves static overdue blocks; accumulates visual debt	Consumes dynamic buffers; applies elastic compression automatically
Visual Feedback	Red error states; high-contrast warning badges	Neutral progress tracking; calm, self-correcting timelines
Estimation Handling	Accepts raw, optimistic estimates; prone to planning fallacy	Applies empirical velocity multipliers based on historical performance
Psychological Framing	Triggers the "What-the-Hell Effect" and user churn	Preserves perceived competence through explicit emergency reserves
Task Model	Fixed absolute time windows ($[t_{\text{start}}, t_{\text{end}}]$)	Elastic spring parameters ($d_i^0, d_i^{\min}, e_i$) solved via Quadratic Optimization

Nuanced Synthesis and System Implementation Guidelines

Computational Engine Architecture

To build a responsive, offline-first productivity system without cloud dependencies, the engine organizes its computational pipeline into three coordinated layers:
The Chronobiological Assessment Engine translates raw user chronotype indicators (wake times, self-reported mid-sleep points, or passive device interactions) into real-time curves for $C(t)$, $S(t)$, and the penalty kernel $\Psi_{\text{dip}}(t)$. These curves generate efficiency profiles across the day, ensuring cognitively demanding analytical tasks are placed during peak executive focus windows while routing low-intensity administrative tasks to periods of circadian vulnerability.
The Reverse Spaced-Milestone Back-Planner evaluates long-range academic commitments (such as exams or essays) and constructs a structured sequence of study blocks. By applying Cepeda's optimal spacing ratios, this component schedules reviews when they provide the greatest memory consolidation, while strictly capping review volume at 20% of daily capacity to prevent timetable congestion.
The Combinatorial Rescheduling Engine manages day-to-day execution and unexpected delays. Using a deterministic, solver-free algorithm, it absorbs schedule slippage through elastic task compression and centralized buffer management. Delays are resolved automatically without user micromanagement, preventing the guilt and frustration that lead to system

abandonment.

Operational Engineering Principles

Building an offline-first scheduling system for high-performing pre-university students requires adhering to four core engineering principles:
First, maintain deterministic, solver-free guarantees. Because mobile environments cannot rely on heavy external solvers like CPLEX or Gurobi, all algorithmic logic must use closed-form heuristics, like the Buttazzo spring compression model, which deliver predictable O(N \log N) or O(N^2) execution times directly on device hardware.
Second, treat fixed commitments as immovable boundaries. Academic timetables include hard constraints (such as classes, laboratories, and athletic commitments) that cannot yield. The engine treats these events as rigid boundaries, focusing all optimization, compression, and deferral on the elastic soft tasks that lie between them.
Third, protect deliberate recovery time. The engine must treat recovery periods—such as the 45-to-60-minute post-school buffer and the 4.5-hour daily limit on deliberate deep work—as primary constraints rather than optional slack. Overworking these boundaries degrades memory consolidation, increases sleep debt, and accelerates mental burnout.
Fourth, design for psychological resilience. By incorporating emergency reserves, silent auto-healing, and adaptive velocity multipliers, the engine ensures that unexpected delays do not cause the entire plan to fail. Preserving the student's self-efficacy and sense of control is essential for sustaining consistent academic performance over the long term.

Works cited

1. Comparison of the Two-Process Model and a Mutual Inhibition, https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0103877 2. The two-process model of sleep regulation: A reappraisal, https://www.researchgate.net/publication/290444035_The_two-process_model_of_sleep_regulation_A_reappraisal 3. (PDF) A unified mathematical model to quantify performance, https://www.researchgate.net/publication/236459149_A_unified_mathematical_model_to_quantify_performance_impairment_for_both_chronic_sleep_restriction_and_total_sleep_deprivation 4. R You Missing SAFTE In Your Next Research Project?, https://www.saftefast.com/safte-fast-blog/r-you-missing-safte-in-your-next-research-project 5. Block diagram of the SAFTE Model. - ResearchGate, https://www.researchgate.net/figure/Block-diagram-of-the-SAFTE-Model_fig1_5337592 6. Interface for a system and method for evaluating task effectiveness, https://patents.google.com/patent/US7118530B2/en 7. Field Validation of Biomathematical Fatigue Modeling - FAA, https://www.faa.gov/sites/faa.gov/files/data_research/research/med_humanfacs/oamtechreports/201212.pdf 8. Mathematical Models for Sleep-Wake Dynamics - arXiv, https://arxiv.org/pdf/1311.1734 9. Validating and Extending the Three Process Model of Alertness in, https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0108679 10. A new mathematical model for the homeostatic effects of sleep loss, https://pmc.ncbi.nlm.nih.gov/articles/PMC2657297/ 11. Attention Restoration Theory: Recovery Through Nature - Pomogolo, https://pomogolo.com/blog/attention-restoration 12. The benefits of mystery in nature on attention: assessing the impacts, https://pmc.ncbi.nlm.nih.gov/articles/PMC4244865/ 13. Attention restoration theory - Wikipedia,

https://en.wikipedia.org/wiki/Attention_restoration_theory 14. The Restorative Benefits of Nature: Toward an Integrative Framework, https://www.researchgate.net/publication/222305048_The_Restorative_Benefits_of_Nature_Toward_an_Integrative_Framework 15. Elastic Scheduling for Graceful Degradation of Mixed-Criticality, https://www.sudvarg.com/publications/RTNS2024_mixed_criticality_elastic.pdf 16. Generalized Elastic Scheduling for Real-Time Tasks - ResearchGate, https://www.researchgate.net/publication/260585324_Generalized_Elastic_Scheduling_for_Real-Time_Tasks 17. Elastic scheduling for flexible workload management - ResearchGate, https://www.researchgate.net/publication/3044454_Elastic_scheduling_for_flexible_workload_management 18. Generalized Elastic Scheduling for Real-Time Tasks, https://www.rtx.ece.vt.edu/resources/Files/chantem09mar.pdf 19. Elastic Scheduling for Flexible Workload Management, https://retis.santannapisa.it/~giorgio/paps/2002/toc02a.pdf 20. Improved Elastic Scheduling Algorithms for Implicit-Deadline Tasks, https://www.sudvarg.com/publications/LITES2025_improved_implicit_elastic.pdf 21. Determining a fuzzy model of time buffer size in critical chain project, https://yadda.icm.edu.pl/baztech/element/bwmeta1.element.baztech-71a592dd-e56b-4b6b-ab05-63a93892e955/c/Kolodziej_Determining_3_24.pdf 22. Critical Chain Project Buffer Sizing Based on Capabilities Attributes, https://www.tandfonline.com/doi/pdf/10.1080/10429247.2025.2464720 23. Buffer Sizing in Critical Chain Project Management by Brittle Risk, https://www.mdpi.com/2075-5309/12/9/1390 24. (PDF) New approach for buffer sizing in Critical Chain scheduling, https://www.researchgate.net/publication/224300333_New_approach_for_buffer_sizing_in_Critical_Chain_scheduling 25. 020-0439 Buffer sizing approach with dependence assumption, https://pomsmeetings.org/confpapers/020/020-0439.pdf 26. Spacing Effects in Learning: A Temporal Ridgeline of Optimal, https://www.researchgate.net/publication/23657355_Spacing_Effects_in_Learning_A_Temporal_Ridgeline_of_Optimal_Retention 27. Optimizing distributed practice online | Studies in Second Language, https://www.cambridge.org/core/journals/studies-in-second-language-acquisition/article/optimizing-distributed-practice-online/C833408A4C3BAD939CA39EA734423BB7 28. Lag effects in grammar learning: A desirable difficulties perspective, https://diposit.ub.edu/bitstreams/c837db24-0ce8-4ec8-bacd-c0c02a6cb215/download 29. Input Scheduling and L2 Grammar Learning: Investigating the, https://ilt.atu.ac.ir/article_21048.html 30. Nudging persistence after failure through emergency reserves, https://ideas.repec.org/a/eee/jobhdp/v163y2021icp17-29.html 31. Katy Milkman on How to Change - Social Science Space, https://www.socialsciencespace.com/2025/02/katy-milkman-on-how-to-change/